Mathematical Modelling and Preliminary Data Processing of LiDAR

An internship report submitted

in partial fulfillment for the award of the degree of

Bachelor of Technology

in

Electronics and Communication Engineering

by

Samedh Sachin Kari

Department of Avionics

Indian Institute of Space Science and Technology

Thiruvananthapuram, India

27 May 2024 - 26 July 2024

Abstract

Light Detecting And Ranging (LiDAR) systems have gained significant popularity in recent years, demonstrating versatile applications across a wide array of fields, including automotive technology, aerospace missions, geoscience, and environmental monitoring. This report provides an in-depth exploration of the fundamental working principles of LiDAR systems, elucidating the underlying technologies that enable accurate distance measurement and high-resolution spatial data collection.

In addition, we present a comprehensive numerical method designed to enhance the understanding of LiDAR operations, which serves as a critical foundation for subsequent analyses. Central to this report is the development of an algorithm aimed at clustering points within a LiDAR point cloud image. This algorithm utilizes proximity and geometric properties to effectively group data points, facilitating a clearer interpretation of the spatial structure inherent in the point cloud.

Moreover, the report addresses the challenge of distinguishing between relevant and irrelevant clusters by implementing a filtering process. This process systematically rejects unwanted clusters based on specific characteristics, ensuring that only pertinent data is retained for further analysis. By improving the clarity and usability of LiDAR data, this study contributes valuable insights into the efficient processing of spatial information, which is essential for advancing applications across various disciplines.

Contents

List of Figures

Abbreviations

LiDAR	Light Detection And Ranging
RADAR	RAdio Detection And Ranging
SONAR	Sound Navigation And Ranging
LASER	Light Amplification by Stimulated Emission of Radiation
TOF	Time of Flight
FOV	Field of View
FDTD	Finite Difference Time Domain

Chapter 1

Introduction

1.1 Internship Overview

This report documents the studies conducted during the internship, which was primarily focused on mathematical modeling of the LiDAR system and preliminary data processing for the estimation of LiDAR image. The primary objectives of this internship were:

1. Mathematical Modeling of LiDAR

2. Preliminary Data Processing Techniques for LiDAR point cloud data

1.2 LiDAR Fundamentals

LiDAR, an acronym for Light Detection and Ranging, is a remote sensing technology that utilizes laser pulses to measure distances to a target. By precisely timing the return of the reflected laser light, LiDAR systems generate highly accurate three-dimensional representations of the environment. This data, often referred to as point clouds, comprises a vast collection of three-dimensional coordinates corresponding to the target. LiDAR finds applications in diverse fields, including autonomous vehicles, space missions, geography and urban planning.

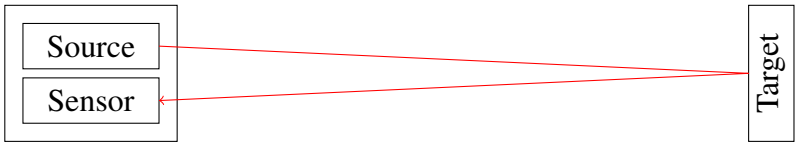


Figure 1.1: Working Principle of LiDAR

LiDAR is essentially an extension of traditional remote sensing techniques like RADAR and SONAR. The fundamental principle underlying all these methods involves the emission of a wave from a source, its reflection off a target, and the subsequent capture of the reflected signal by a sensor. By calculating the time of flight, which is the duration between the wave's emission and its reception, the distance to the target can be determined, as shown in Figure 1.1. Analyzing other characteristics of the received signal, in conjunction with the range, enables the extraction of additional properties about the target.

1.3 Flash LiDAR

LiDARs use laser as the source which is diverged to illuminate a target area. Based on the mechanism of this, LiDARs can be classified into several types. Flash LiDAR is one of the types which illuminaltes the target area by creating a flash. A flash has coherent and monochromatic light, which means the light has a single frequency and constant phase. The flash LiDAR doesn't have any moving parts. The laser beam is passed through an optical diffuser that causes angular diffusion to it. The working of Flash LiDAR is shown in Figure 1.2.

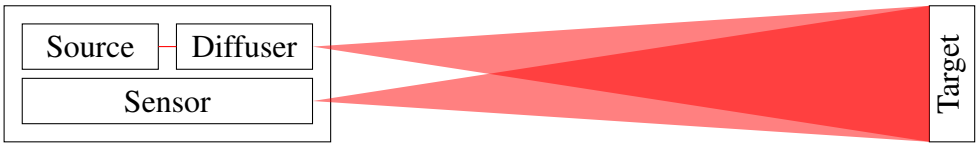


Figure 1.2: Flash LiDAR

Hence, the mathematical model of the Flash LiDAR can be divided into several parts like modelling the sensor, diffuser, propagation of electromagnetic waves, reflection of waves at the target, and the receiver.

1.4 Document Structure

This document is structured into three main chapters. Chapter 2 delves into the physical phenomena and associated mathematics governing LiDAR operations, including its mathematical model. Chapter 3 discusses the transformation of this model into a numerical method and its implementation as a computer program. Chapter 4 details the initial data processing techniques used for LiDAR image processing and estimating the target's center

of mass within the image. Lastly, Chapter 5 concludes the report. The codes involved are given at the end as appendices.

Chapter 2

Principles of Operation of LiDAR

To construct a mathematical model and implement it as a computer program, understanding of the underlying physics governing the phenomenon is crucial. Therefore, the initial focus of the internship was to understand the essential physical principles employed in LiDAR technology, detailed in this section.

2.1 Laser: The Source

The LiDAR system uses laser as its primary source. Laser produces radiation which have a significantly high degree of directionality, coherence, monochromaticity and brightness [1]. This prompted further exploration into the operational principles of lasers.

2.1.1 Laser as an Oscillator

A laser functions as an oscillator since photons oscillate continuously within its cavity. Figure 2.1 shows the model of a laser. An active medium is present in the cavity which is responsible for releasing photons which contribute to the rise in flux of photons. This active medium is supported by a pump system that provides power to it. The laser cavity is bounded by two mirrors, one highly reflective and the other partially reflective. The partially reflective mirror allows a fraction of incident photons to escape the laser cavity. These photons act as the laser beam which is the source of our LiDAR.

In the cavity, there is an oscillating photon flux. Assume that it undergoes an attenuation denoted by α. The active medium contributes a gain, G, to this photon flux. Assume that the mirrors have reflectivities of R_1 and R_2, respectively. Now to have sustainable oscillations, it should satisfy the condition

$$(1-\alpha)\, G R_1 R_2 \geq 1 \tag{2.1}$$

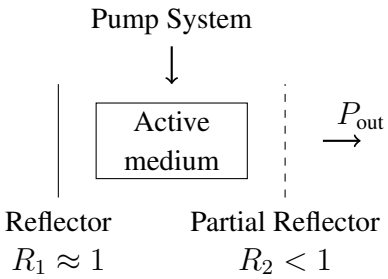
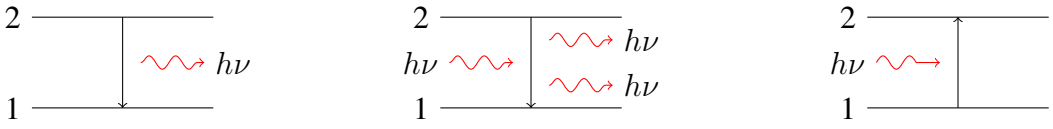


Figure 2.1: Model of a laser

If this condition is not satisfied, the oscillations will eventually die out. Hence it should be ensured that corresponding gain is achieved to have sustainable oscillations.

2.1.2 Electron Transitions

The active medium provides gain by emitting photons which contribute to the flux. The pump system provides energy which is used to excite the electrons in this active medium to higher energy states. The electrons return back to their base state by releasing energy in the form of photons. Thus, it is understood that the electron transitions occurring within the active medium serve as the driving force for the laser. Further exploration into these electronic transitions follows.



(a) Spontaneous Emission **(b)** Stimulated Emission **(c)** Absorption

Figure 2.2: Electron Transitions

Absorption, spontaneous emission, and stimulated emission of photons, along with the corresponding electron transitions, are the three processes that occur in an atom, which are shown in the Figure 2.2. In all these transitions, energy will be conserved. For example if an electron comes from higher energy state to a lower energy state and releases a photon, then energy of the photon will be equal to the energy lost by the electron, or $E_2 - E_1 = h\nu$, where E_2 and E_1 are energies of higher and lower energy level respectively, h is Plank's constant and ν is the frequency of photon.

Spontaneous emission is when an electron drops from a higher energy state to a lower one emitting a photon. This photon need not have the same direction or a definite phase

relations with other spontaneously emitted photons. The rate of spontaneous emission is proportional to the population of electrons in the excited state. Hence the rate of spontaneous emission is given by

$$\left(\frac{dN_2}{dt}\right)_{sp} = -AN_2 \tag{2.2}$$

where N_2 is the population of electrons in level 2 and A is a coefficient which has unit s^{-1} and depends on the transition.

When a photon of frequency same as the frequency corresponding to the transition initiates this transition of electron from level 2 to level 1, the process is known as stimulated emission. A photon emitted by stimulated emission has same frequency, phase and direction as that of the incident photon. The rate of stimulated emission of photons is given by

$$\left(\frac{dN_2}{dt}\right)_{st} = -W_{21}N_2 \tag{2.3}$$

where N_2 is the population of electrons in level 2 and W_{21} is a coefficient whose value depends on the transition as well as the incident photon. Hence W_{21} can be expressed as

$$W_{21} = \sigma_{21}F \tag{2.4}$$

where F is the flux of photons and σ_{21} is stimulated emission cross-section having the dimension of area.

The process opposite to stimulated emission is absorption, where a photon is absorbed causes the excitation of electron from level 1 to level 2. The rate of absorption of photons is given by

$$\left(\frac{dN_1}{dt}\right)_{a} = -W_{12}N_1 \tag{2.5}$$

where N_1 is the population of electrons in level 1 and W_{12}, just like W_{21}, is a coefficient whose value depends on the transition as well as the incident photon. Hence W_{12} can be expressed as

$$W_{12} = \sigma_{12}F \tag{2.6}$$

where, F is the flux of photons and σ_{12} is absorption cross-section having the dimension of an area. If energy levels 1 and 2 have g_1-fold and g_2-fold degeneracy respectively, then σ_{21} and σ_{21} are related as

$$g_1\sigma_{12} = g_2\sigma_{21} \tag{2.7}$$

2.1.3 Gain

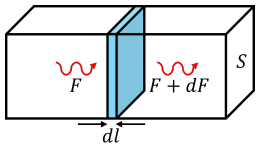


Figure 2.3: Gain provided by active medium

It is known that the photons emitted in the active medium constitute as the gain in flux, which needs to be calculated. Consider a volume of active medium with surface area S as shown in Figure 2.3. Let photons travel from one end of the cavity to the other, so they pass through the active medium once. Consider a slice of infinitesimal thickness dl which has flux of photons F entering and an increased flux of photons $F + dF$ leaving the this slice of active medium. The rate of photons produced in this slice can be given by SdF which can also be written as $S(W_{21}N_2 - W_{12}N_1)dl$. Equating these expressions and integrating, the obtained equation is

$$SdF = S(W_{21}N_2 - W_{12}N_1)dl \tag{2.8}$$

Then Equations 2.4, 2.6 and 2.7 can be substituted and integrated to get the gain, G:

$$G = \frac{F_{final}}{F_{initial}} = \exp\left[\sigma_{21}\left(N_2 - \frac{g_2}{g_1}N_1\right)l\right] \tag{2.9}$$

where l will be the length of the active medium. Here σ_{21}, g_1, g_2, l are all constants for a given laser. Hence gain will be a function of N_1 and N_2 which vary with time.

2.1.4 Steady State

When a laser is off, it will not have any flux of photons in the cavity and majority of the electrons will be present in the lower energy state. However, after turning the laser on, N_2 will start rising and N_1 will fall proportionately. Initially the gain will be very high, but it will reduce eventually. The flux will rise until steady state is achieved. At steady state, N_1 and N_2 will stabilise as emission will be balanced by absorption. At steady state, power present in the cavity and the output power will be constant, which corresponds to equality in Equation 2.1. Substituting Equation 2.9 in Equation 2.1 for the condition of equality, it is found that the threshold value of $(N_2 - (g_2/g_1)N_1)$ equal to

$$\left(N_2 - \frac{g_2}{g_1}N_1\right)_{th} = -\frac{[\ln R_1R_2 + 2\ln(1-\alpha)]}{2\sigma_{21}l} \tag{2.10}$$

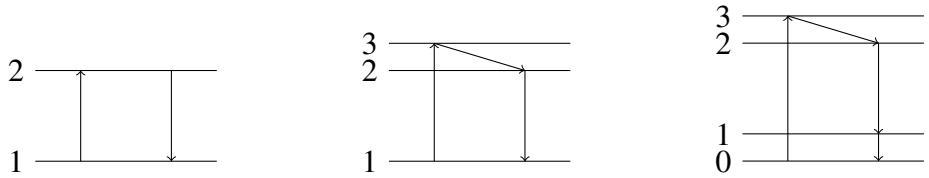
2.1.5 Inversion

In Equation 2.10, the value of $(N_2 - (g_2/g_1)N_1)$ at steady state was found. However, on analysing the equation, it is realised that R_1R_2 is the product of reflectivities, which will be less than unity, as one of the mirror reflects partial light. $(1 - \alpha)$, which is derived from the attenuation factor is also less than unity. In the numerator, the $\log$ of these expressions is present, which will be negative. That would make overall right side of the equation positive.

Hence, at steady state condition, $(N_2 - (g_2/g_1)N_1)$ should be positive, which implies $N_2 \geq (g_2/g_1)N_1$. This effectively means that at steady state, the population of electrons at level 2 should be greater than population of electrons at level 1. This is known as inversion, and it is a necessity for laser a laser to function at steady state.

2.1.6 Mechanism of Active Medium

As it is known that inversion is a necessity for operation of a laser, let's understand more about achieving inversion [2].



(a) 2 level **(b)** 3 level **(c)** 4 level

Figure 2.4: Multilevel Systems

First, a 2 level system is considered as shown in Figure 2.4a. Here electrons are excited from level 1 to level 2 by absorption of a photon, followed by stimulated or spontaneous emission of the photon. For simplicity, it is assumed that these 2 levels have same degeneracy, i.e., $W_{21} = W_{12} = W$. Since it is known that absorption, spontaneous emission and stimulated emission are the only processes governing the population of electrons in the 2 levels, the rates of change of population of electrons in these levels can be found, which are

$$\frac{dN_2}{dt} = W\left(N_1 - N_2\right) - AN_2 \tag{2.11a}$$

$$\frac{dN_1}{dt} = W\left(N_1 - N_2\right) + AN_2 \tag{2.11b}$$

Now the notations $N = N_1 + N_2$ and $\Delta N = N_1 - N_2$ are used. Subtracting Equation 2.11b from 2.11a, the equation obtained is

$$\frac{d\Delta N}{dt} = -2W\Delta N + AN - A\Delta N \tag{2.12}$$

However, when the laser attains steady state, N_1 and N_2 won't change as absorption will be balanced by emission. Hence $d\Delta N/dt = 0$. Substituting this in Equation 2.12, the following equation is obtained

$$\Delta N = \frac{AN}{2W + A} \tag{2.13}$$

Since ΔN is positive over here, $N_1 > N_2$, which means inversion can never be achieved in a 2 level system.

Now, let's consider a 3 level system, as shown in Figure 2.4b. Here the electron is excited from level 1 to level 3, after which it undergoes a fast decay and comes to level 2. Then it moves from level 2 to level 1 emitting a photon. The rate equations for this system will be

$$\frac{dN_1}{dt} = W_{13}N_3 + W_{12}N_2 + A_{21}N_2 - W_{31}N_1 - W_{12}N_1 \tag{2.14a}$$

$$\frac{dN_2}{dt} = S_{32}N_3 + W_{12}N_1 - A_{21}N_2 - W_{21}N_2 \tag{2.14b}$$

$$\frac{dN_3}{dt} = W_{31}N_1 - W_{13}N_3 - S_{32}N_3 \tag{2.14c}$$

where S is similar to A, but it represents the coefficient for a non-radiating transition [3]. At steady state, all the above rates will be zero, and that could be used to find the values of N_1, N_2 and N_3. These values will give us

$$\frac{N_1 - N_2}{N_1 + N_2} = \frac{A_{21}S_{32} - (S_{32} - A_{21})W_{31}}{2W_{21}(W_{31} + S_{32}) + W_{31}S_{32} + W_{31}A_{21} + S_{32}A_{21}} \tag{2.15}$$

But transition of electron from level 3 to level 2 is a rapid process compared to transition from level 2 to level 1. Hence, $S_{32} \gg A_{21}$. This approximation simplifies our equation to

$$N_1 - N_2 = \frac{A_{21} - W_{31}}{A_{21} + W_{31}} (N_1 + N_2) \tag{2.16}$$

From this equation the possibility of inversion is clearly seen when $W_{31} > A_{21}$. On doing the same analysis for 4 level system, it can be found that inversion is possible for the 4 level system.

2.1.7 Saturation

The 3 level system which was analysed in the previous section will be on the verge of inversion when $W = \sigma F = A$, i.e., when $F = A/\sigma$. The intensity corresponding to this flux is called the saturation intensity, and it is given by

$$I_{sat} = \frac{Ah\nu}{\sigma} = \frac{h\nu}{\sigma\tau} \tag{2.17}$$

where σ is the absorption cross-section and $1/\tau$ is the relaxation rate due to spontaneous emission. Saturation intensity is important because it is the intensity which corresponds to one photon incident on each molecule.

2.1.8 Power

When a laser is turned off, no power is pumped into it and no power is given out by it. As the pump power is increased, initially, the power is used to excite electrons and there is no effective output at steady state, but the flux of photons inside the cavity is rising. When the concentration difference reaches threshold value given in Equation 2.10, the output power starts to rise. The output power is given by

$$P_{out} = SI_{sat}\left(-\frac{ln(1 - R_1R_2)}{2}\right)\left[\frac{P_P}{P_{th}} - 1\right] \tag{2.18}$$

where S is the cross-section area of the active medium, P_p is the pumping power and P_{out} is output power. The electron population difference and output power are shown in Figure 2.5.

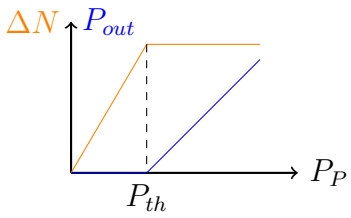


Figure 2.5: Plot for output power and electron population difference

2.2 Electromagnetic Radiations

The previous section discussed the working principle of a laser. The output radiation produced by the laser is an electromagnetic wave. This section delves deeper into electromagnetic waves and radiation.

2.2.1 Light as an Electromagnetic Wave

Light is an electromagnetic wave. It is a wave of alternating electric and magnetic fields which are perpendicular to each other. The behaviour of these electromagnetic fields are governed by Maxwell's equations, which are

$$\nabla \cdot \vec{E} = \frac{\rho_e}{\varepsilon_0} \tag{2.19a}$$

$$\nabla \cdot \vec{B} = 0 \tag{2.19b}$$

$$\nabla \times \vec{E} = -\frac{\partial \vec{B}}{\partial t} \tag{2.19c}$$

$$\nabla \times \vec{B} = \mu_0 \vec{J} + \mu_0 \varepsilon_0 \frac{\partial \vec{E}}{\partial t} \tag{2.19d}$$

where $\vec{E}$ is the electric field and $\vec{B}$ is the magnetic field, ρ_e is the electric charge density, and $\vec{J}$ is the current density. ε_0 and μ_0 are constants for permittivity and permeability of free space.

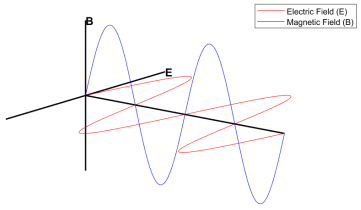


Figure 2.6: Electromagentic Wave

On solving these equations in 1 dimension, the following equation is obtained.

$$\vec{E} = \vec{E_0} \cos\left(\vec{k} \cdot \vec{r} - \omega t\right) \tag{2.20a}$$

$$\vec{B} = \vec{B_0} \cos\left(\vec{k} \cdot \vec{r} - \omega t\right) \tag{2.20b}$$

where $\vec{k}$ is the wave propagation vector with magnitude $2\pi/\lambda$ and direction same as the direction of propagation of wave, $\vec{r}$ is the position vector, ω is the angular frequency of the wave and t denotes the time. A visual representation of this wave is shown in Figure 2.6. As the wave travels, it carries this energy along with it. The energy flux density (energy per unit area per unit time) transported by the fields is given by the Poynting vector, $\vec{S}$ given by

$$\vec{S} = \frac{1}{\mu_0}\left(\vec{E} \times \vec{B}\right) \tag{2.21}$$

The average value of Poynting vector with respect to time, also known as irradiance of light, I, can be calculated, which is

$$I = \frac{1}{2} c\varepsilon_0 E_0^2 \tag{2.22}$$

where c is the speed of light, and E_0 is the peak magnitude of electric field. Irradiance multiplied by the area of cross section of the beam gives us the power of the light beam.

2.2.2 Oscillating Dipole

The most fundamental way of modelling generation of light, which effectively is an electromagnetic radiation, is by oscillation of a dipole [4]. For example, consider a dipole as shown in Figure 2.7.

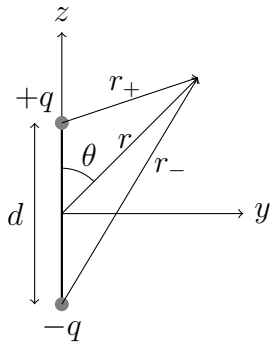


Figure 2.7: Dipole

In Figrure 2.7, value r_+ and r_- can be evaluated using law of cosines, and found to be

$$r_\pm = \sqrt{r^2 \mp rd\cos\theta + (d/2)^2} \tag{2.23}$$

This can be simplified by using the approximation $d \ll c/\omega \ll r$. The electric scalar

potential will be

$$\begin{aligned} V &= \frac{q_0}{4\pi\varepsilon_0}\left\{\frac{\cos\left[\omega(t-r_+/c)\right]}{r_+} - \frac{\cos\left[\omega(t-r_-/c)\right]}{r_-}\right\} \\ V &= -\frac{p_0\omega}{4\pi\varepsilon_0 c}\left(\frac{\cos\theta}{r}\right)\sin\left[\omega(t-r/c)\right] \end{aligned} \tag{2.24}$$

Here the $(t-r/c)$ accounts the state of dipole at the time the of creating concerned potential and not the present state of the dipole. $(t-r/c)$ is known as the retarded time.

Similarly, the effective current between the dipole can be calculated as $\vec{I} = dq/dt\hat{z}$, which will give us the magnetic vector field

$$\vec{A} = -\frac{\mu_0 p_0\omega}{4\pi r}\sin\left[\omega(t-r/c)\right]\hat{z} \tag{2.25}$$

Using these potentials, the electric field obtained is

$$\begin{aligned} \vec{E} &= -\nabla V - \frac{\partial\vec{A}}{\partial t} \\ \vec{E} &= -\frac{\mu_0 p_0\omega^2}{4\pi}\left(\frac{\sin\theta}{r}\right)\cos\left[\omega(t-r/c)\right]\hat{\theta} \end{aligned} \tag{2.26}$$

and magnetic field as

$$\begin{aligned} \vec{B} &= \nabla\times\vec{A} \\ \vec{B} &= -\frac{\mu_0 p_0\omega^2}{4\pi c}\left(\frac{\sin\theta}{r}\right)\cos\left[\omega(t-r/c)\right]\hat{\phi} \end{aligned} \tag{2.27}$$

After obtaining the expressions for electric and magnetic fields, using Equation 2.21, the Poynting Vector for the oscillating dipole can be found.

$$\vec{S} = \frac{\mu_0}{c}\left\{\frac{p_o\omega^2}{4\pi}\left(\frac{\sin\theta}{r}\cos\left[\omega\left(t-r/c\right)\right]\right)\right\}^2\hat{r} \tag{2.28}$$

Time average for the period of one oscillation gives the irradiance of the radiation, I as

$$I = \left(\frac{\mu_0 p_0^2\omega^4}{32\pi^2 c}\right)\frac{\sin^2\theta}{r^2}\hat{r} \tag{2.29}$$

which can be integrated over area to get the power, P, for the radiations produced as

$$P = \frac{\mu_0 p_0^2 \omega^4}{12\pi^2 c} \tag{2.30}$$

Oscillating dipole is the most fundamental way to produce electromagnetic waves. In the laser, excitation and de-excitation of electrons can be modelled as oscillating dipoles, and corresponding output power will be the same as the output power of the laser obtained in Equation 2.18. While implementing the source in the computer program, it will be modelled using the average value of power.

2.3 Optical Diffuser

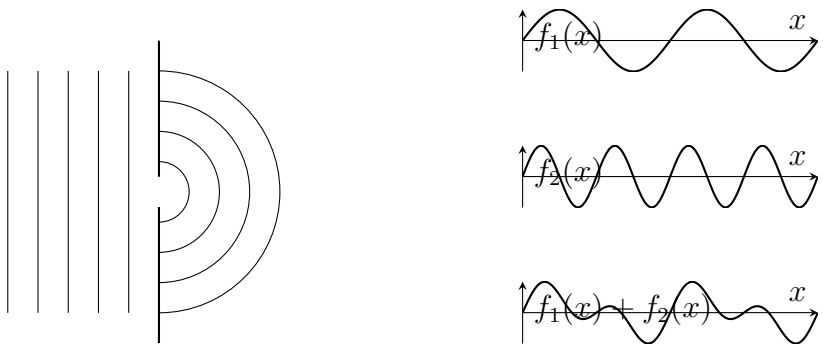
Having understood the working of a laser, which is the source of LiDAR, electromagnetic wave nature of the light, and oscillating dipole, which is the fundamental way to produce these radiations, the next part of the LiDAR system, which is the optical diffuser, needs to be studied. The diffuser is the component which diverges the laser beam to create a flash that would illuminate an area. The diffuser works on the principles of diffraction and interference of light.

2.3.1 Diffraction and Interference

Light usually follows a straight path, which is called rectilinear propagation of light. However, if light encounters an obstacle or aperture during propagation, it shows deviation from rectilinear propagation. In other words, if light passes across an obstacle or aperture, is doesn't travel in a straight line and tends to bend its path. This phenomenon is known as diffraction and it is shown in Figure 2.8a.

Diffraction can be explained by Huygens–Fresnel Principle of wave propagation. This principle explains the propagation of waves in the form of wavefronts where every point on the wavefront acts as secondary source producing spherical wavelets [5]. If such a wave encounters a very small slit, only a few points on the wavefront will produce secondary wavelets that would propagate further. As a result, spherical wavefronts are obtained from a a wave which was initially planar. If the slit is bigger, there will be multiple spherical wavelets produced, and that would result in a planar wavefront corresponding to the area of the slit.

Optical interference corresponds to the interaction of two or more light waves yielding



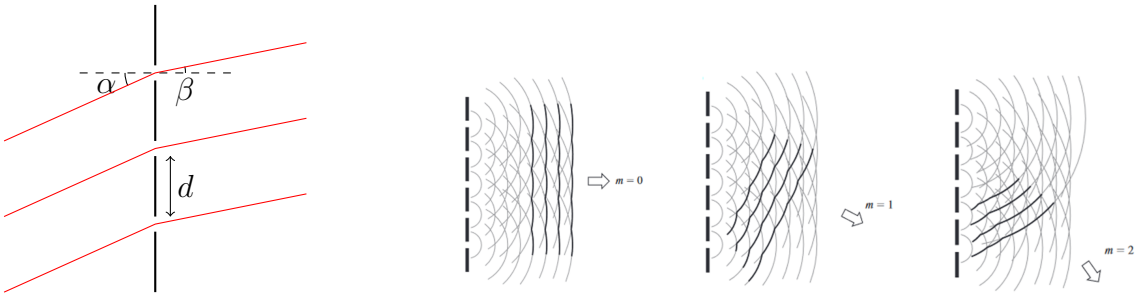
(a) Diffraction **(b)** Interference

Figure 2.8: Optical Phenomena

a resultant irradiance. The resultant irradiance is the superposition of irradiance of the contributing light waves. Figure 2.8b shows the optical interference of two waves. Interference of multiple diffracted waves causes several patterns governed by the path difference of the waves. This is explored in the next section.

2.3.2 Diffraction Grating

A diffraction grating is a repeating pattern of diffractive elements, such as apertures or periodic structure of different optical material. Each diffractive element acts as an individual source of scattered light, collectively forming a uniform array of linear sources. As a result, the emerging wavefront exhibits periodic changes in its shape, corresponding to an angular distribution of component plane waves.



(a) Diffraction Grating **(b)** Grating Output[5]

Figure 2.9: Diffraction Grating

The working principle of diffraction grating is shown in Figure 2.9a, where light is incident on a grated surface at an angle of α and it leaves the surface at an angle β. The repetition length for the grated surface is d. The path difference between the incident and

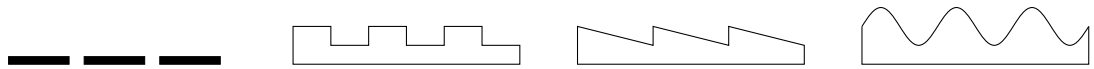


Figure 2.10: Types of Diffraction Grated Surfaces

reflected wavefront should be a multiple of wavelength. This gives us the grating equation

$$m\lambda = d\left(\sin\alpha + \sin\beta\right) \tag{2.31}$$

where m is a whole number and λ is the wavelength of the light. If the incident angle, $\alpha = 0$, then

$$\beta\left(\lambda\right) = \sin^{-1}\left(\frac{m\lambda}{d}\right) \tag{2.32}$$

Grating a surface can direct the output waveforms in multiple directions which can be intuitively understood from Figure 2.9b. The phase difference between light from two consecutive slits is $(2\pi(d/\lambda)sin\theta)$. Hence, for n slits, the ratio of intensity in direction at an angle θ with respect to normal to the grating surface to that in the direction normal will be

$$\frac{I(\theta)}{I_0} = \left(\frac{E(\theta)}{E_0}\right)^2 = \left(\frac{\sin\left(\pi N(d/\lambda)\sin\theta\right)}{\sin\left(\pi(d/\lambda)\sin\theta\right)}\right)^2 = \frac{\sin^2\left(\pi N(d/\lambda)\sin\theta\right)}{\sin^2\left(\pi(d/\lambda)\sin\theta\right)} \tag{2.33}$$

Figure 2.10 shows several ways of grating surfaces. It either uses a surface with slits or a material of different refractive index with a repeating geometry.

2.4 Receiver array

After studying all the aspects of the source, the properties of the receiver need to be studied. The receiver used by the LiDAR system is a 2 dimensional array of photodiodes. These photodiodes are semiconductor devices that produce current that is proportional to the intensity of the incident light. Usually avalanche photodiodes are used to serve this purpose.

2.4.1 Avalanche Photodiodes

An avalanche photodiode is a type of photodetector which is highly responsive to light. It has a highly doped p-type section (p^+) at one end with acceptor charge density N_a^+,

followed by an intrinsic region, a regularly doped p-type section (p) with acceptor charge density N_a, and a highly doped n-type section (n^+) [6]. Such a photodiode and the variation in charge density and electric field in the diode is shown in Figure 2.11.

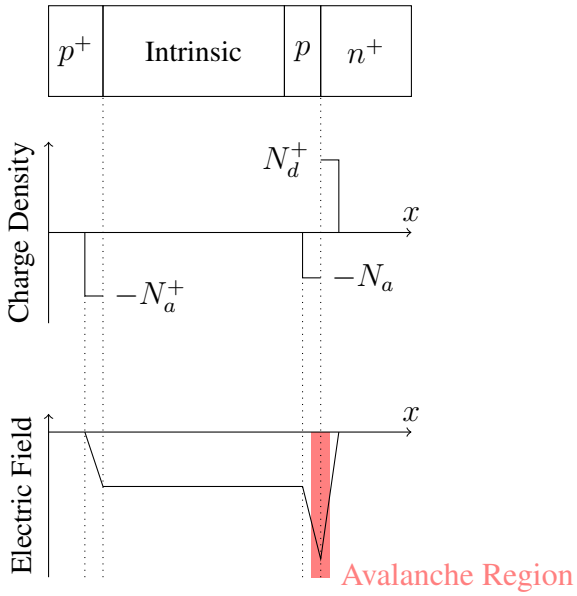


Figure 2.11: Avalanche Photodiode

The avalanche photodiode operates by applying a reverse bias voltage, creating high electric field in the avalanche region. When photons enter the intrinsic region, it generates electrons that drift through the avalanche area. With sufficient voltage, these electrons gain kinetic energy and multiply by knocking loose additional electrons from neutral pairs. In essence, the electric current is amplified at the avalanche region.

The response of the photodiode is determined using the Quantum Efficiency (η) of the intrinsic medium, which is the ratio of number of electrical carriers generated and the number of photons injected. If n photons are incident on the diode during an interval of Δt, then the optical power, $P = n \cdot h\nu/\Delta t$, and the photocurrent generated, $I = Mn\eta q/\Delta t$, where M is the gain factor. The responsivity of the diode, $\mathcal{R}$, which is the ratio of photocurrent and optical power, is given by

$$\mathcal{R} = \frac{I}{P} = M\eta\frac{q}{h\nu} \tag{2.34}$$

where M is the gain of the photodiode and is usually calculated as

$$M = \frac{1}{1 - (V_B/V_{BD})^{n_B}} \tag{2.35}$$

where n_B is a parameter that depends on the structure and material of the device.

2.4.2 3D Point Cloud Generation

Data captured by a LiDAR is stored as a point cloud. As studied earlier, the sensor used in a LiDAR is a 2-dimensional array of photodetectors. Each diode in the photodetector array records the time of flight (TOF) of the laser pulse. Using this data along with the geometrical properties of the sensor and the optical properties of the focal lens used in the receiver, the x, y, and z coordinate for every diode in the array can be calculated and stored as a point cloud.

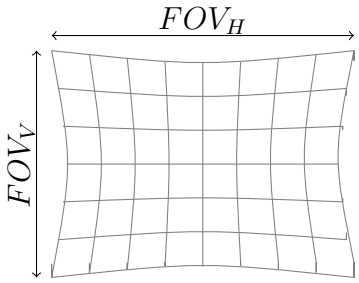


Figure 2.12: Field of View

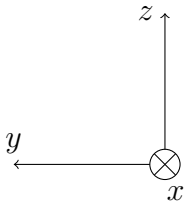


Figure 2.13: Coordinate System

Initially, the distance of the target from the source, r, is calculated which will be given by

$$r = \frac{TOF \times c}{2} \tag{2.36}$$

where TOF is the time difference between the laser signal emission and the signal reception at the sensor and c is the speed of light. Then the the horizontal and vertical field of view of the sensor can be calculated. These are given by

$$FOV_H = 2\tan^{-1}\left(\frac{w_{\text{sensor}}}{2f}\right) \tag{2.37a}$$

$$FOV_V = 2\tan^{-1}\left(\frac{h_{\text{sensor}}}{2f}\right) \tag{2.37b}$$

where w and h are the width and the height of the photodetector array and f is the focal length of the lens used at the receiver. If the array has $p \times q$ diodes then the 2D FOV is divided into grid of angles as shown in Figure 2.12. Using r, and the horizontal and vertical angles of a particular diode, θ_H and θ_V, coordinates corresponding to every angle can be

calculated as follows. The coordiantes follow the coordinate system shown in Figure 2.13.

$$\begin{aligned} x &= r\cos\theta_H\cos\theta_V \\ y &= r\sin\theta_H\cos\theta_V \\ z &= r\sin\theta_V \end{aligned} \tag{2.38}$$

The data recorded by the LiDAR is stored as a point cloud along with other parameters recorded by the sensors like intensity. The format usually used to store the LiDAR data `.las` format [7].

2.4.3 Target Properties

After determining the position of points in the point cloud, further properties of the target are found. LiDARs usually sense the intensity of each point in the point cloud along with the position. This intensity is a function of the reflectivity, incident angle and distance to the target, which can be expressed as a product of corresponding independent functions, expressed by

$$I\left(\rho,\theta,R\right) = C_o \cdot f_\rho(\rho) \cdot f_\theta(\theta) \cdot f_R(R) \tag{2.39}$$

where ρ is the reflectivity of the target surface, θ is the angle of incidence with respect to the target surface, and R is the distance between the sensor and target [8]. For a perfectly Lambertian surface, $f_\rho(\rho) = \rho$, $f_\theta(\theta) = \cos\theta$ and $f_R(R) = 1/R^2$. However, for non-ideal surfaces, $f_\rho(\rho), f_\theta(\theta), f_R(R)$ can be determined by performing iterations while changing one parameter.

Chapter 3

Simulation Architecture

3.1 Finite Difference Time Domain Method

The Finite Difference Time Domain (FDTD) Method stands out as the widely adopted technique for electromagnetic wave simulation. It utilizes finite differences to approximate the spatial and temporal derivatives found in Maxwell's equations, namely Equations 2.19c and 2.19d. Initially introduced by Kane Yee in 1966, the FDTD algorithm employs second-order central differences for its computations [9]. An overview of the Yee Algorithm is given in Figure 3.1

3.1.1 Obtaining Difference Equations

So, to implement the algorithm, first Maxwell's Equations 2.19c and 2.19d need to be discretized. This was done in 2 dimensions in TM^z mode, where the wave travels in the xy plane with it's electric field in the z direction and magnetic field is in the xy plane. $\vec{J} = \sigma\vec{E}$ is substituted to get the following equations.

$$-\frac{\partial \vec{B}}{\partial t} = \nabla \times \vec{E} = \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ 0 & 0 & E_z \end{vmatrix} = \frac{\partial E_z}{\partial y}\hat{x} - \frac{\partial E_z}{\partial x}\hat{y} \tag{3.1a}$$

$$\mu_0\sigma\vec{E} + \mu_0\varepsilon_0\frac{\partial \vec{E}}{\partial t} = \nabla \times \vec{B} = \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & 0 \\ B_x & B_y & 0 \end{vmatrix} = \left(\frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y}\right)\hat{z} \tag{3.1b}$$

However, since the ratio of the magnitudes of $\vec{E}$ and $\vec{B}$ corresponds to the speed of light, which is quite large, calculations involving these two quantities can be challenging. There-

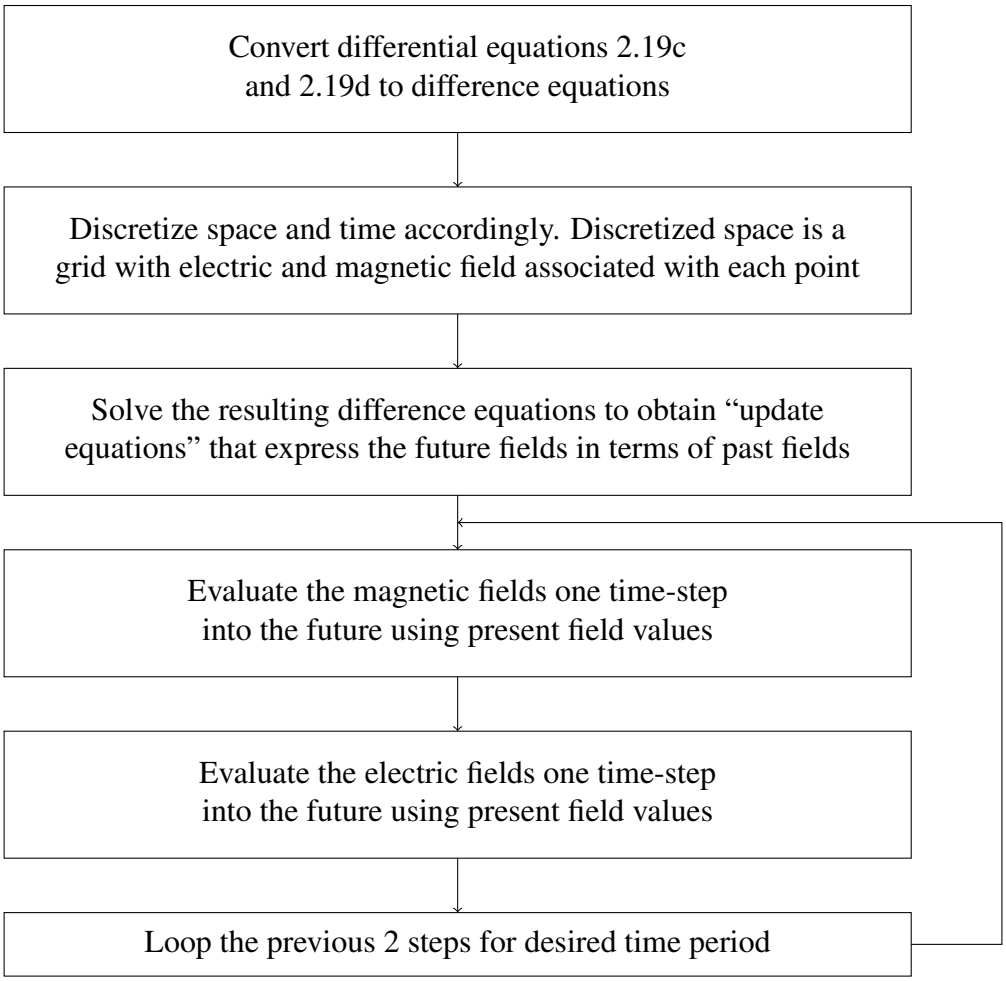


Figure 3.1: Overview of the FDTD Algorithm

fore, $\vec{H} = \vec{B}/\mu$ can be used instead of $\vec{B}$ in this context. This substitution will reformulate the equations as follows

$$-\mu_0 \frac{\partial \vec{H}}{\partial t} = \nabla \times \vec{E} = \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ 0 & 0 & E_z \end{vmatrix} = \frac{\partial E_z}{\partial y}\hat{x} - \frac{\partial E_z}{\partial x}\hat{y} \tag{3.2a}$$

$$\sigma \vec{E} + \varepsilon_0 \frac{\partial \vec{E}}{\partial t} = \nabla \times \vec{H} = \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & 0 \\ H_x & H_y & 0 \end{vmatrix} = \left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} \right) \hat{z} \tag{3.2b}$$

Solving these equations gives 3 scalar equations, one in each direction.

$$\mu_0 \frac{\partial H_x}{\partial t} = \frac{\partial E_z}{\partial y} \tag{3.3a}$$

$$\mu_0 \frac{\partial H_y}{\partial t} = \frac{\partial E_z}{\partial x} \tag{3.3b}$$

$$\sigma E_z + \varepsilon_0 \frac{\partial E_z}{\partial t} = \frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} \tag{3.3c}$$

These differential equations can now be converted into difference equations. Assume that the variables x, y, t are replaced by X, Y, T for the difference equations. Then these equations become

$$\mu_0 \frac{H_x[X,Y,T] - H_x[X,Y,T-1]}{\Delta T} = \frac{E_z[X,Y,T] - E_z[X,Y-1,T]}{\Delta Y} \tag{3.4a}$$

$$\mu_0 \frac{H_y[X,Y,T] - H_y[X,Y,T-1]}{\Delta T} = \frac{E_z[X,Y,T] - E_z[X-1,Y,T]}{\Delta X} \tag{3.4b}$$

$$\sigma \left(\frac{E_z[X,Y,T] + E_z[X,Y,T-1]}{2} \right) + \varepsilon_0 \left(\frac{E_z[X,Y,T] - E_z[X,Y,T-1]}{\Delta T} \right) = \frac{H_y[X,Y,T] - H_y[X-1,Y,T]}{\Delta X} - \frac{H_x[X,Y,T] - H_x[X,Y-1,T]}{\Delta Y} \tag{3.4c}$$

3.1.2 Discretize Space and Time

Space will be discretized as 2 dimensional grid in this case, as the electromagnetic waves are implemented in 2 dimensions. However, since the electric and magnetic fields are updated on alternate iterations, the difference equations won't be applicable on the boundaries of the grid. To address this the FDTD algorithm follows a convention of having electric field nodes on the boundaries of the grids. Such a grid is shown if Figure 3.2. Circles in the grid represent the electric field and triangles represent the magnetic fields.

As seen in Figure 3.2, if the E_z matrix has size $m \times n$, then H_x and H_y matrices will have sizes $(m-1) \times n$ and $m \times (n-1)$ respectively.

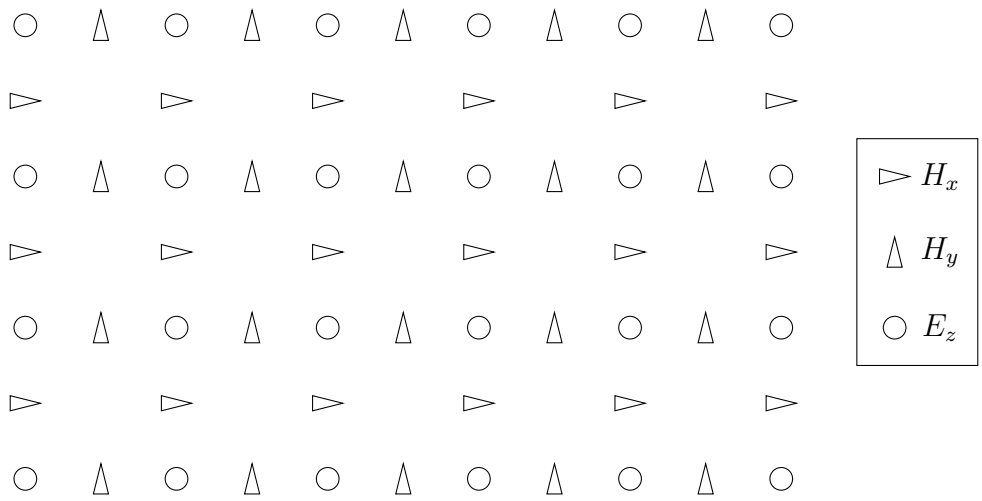


Figure 3.2: FDTD Grid

3.1.3 Update Equations

Rearranging the terms in Equations 3.4, the value for $H_x[X,Y,T], H_y[X,Y,T], E_z[X,Y,T]$ can be obtained as

$$H_x[X,Y,T] = H_x[X,Y,T-1] + \frac{\Delta T}{\mu_0 \Delta Y}\left(E_z[X,Y,T] - E_z[X,Y-1,T]\right) \tag{3.5a}$$

$$H_y[X,Y,T] = H_y[X,Y,T-1] + \frac{\Delta T}{\mu_0 \Delta X}\left(E_z[X,Y,T] - E_z[X-1,Y,T]\right) \tag{3.5b}$$

$$E_z[X,Y,T] = \left(\frac{2\varepsilon_0 - \sigma\Delta T}{2\varepsilon_0 + \sigma\Delta T}\right) E_z[X,Y,T-1] + \frac{2\varepsilon_0}{2\varepsilon_0 + \sigma\Delta T} \left(\frac{\Delta T}{\varepsilon_0 \Delta X}(H_y[X,Y,T] - H_y[X-1,Y,T]) - \frac{\Delta T}{\varepsilon_0 \Delta Y}(H_x[X,Y,T] - H_x[X,Y-1,T])\right) \tag{3.5c}$$

These equations can be simplified further by handling the constants in a better way. Firstly it is assumed that the grid is symmetric in both dimensions, which implies $\Delta X = \Delta Y$. Then the terms $\Delta T/\varepsilon_0 \Delta X$ and $\Delta T/\mu_0 \Delta X$ can be written as

$$\frac{1}{\varepsilon_0}\frac{\Delta T}{\Delta X} = \frac{\sqrt{\varepsilon_0 \mu_0}}{\varepsilon_0}\frac{c\Delta T}{\Delta X} = \sqrt{\frac{\mu_0}{\varepsilon_0}}\frac{c\Delta T}{\Delta X} = \eta_0 \frac{c\Delta T}{\Delta X} = \eta_0 S_c \tag{3.6a}$$

$$\frac{1}{\mu_0}\frac{\Delta T}{\Delta X} = \frac{\sqrt{\varepsilon_0\mu_0}}{\mu_0}\frac{c\Delta T}{\Delta X} = \sqrt{\frac{\varepsilon_0}{\mu_0}}\frac{c\Delta T}{\Delta X} = \frac{1}{\eta_0}\frac{c\Delta T}{\Delta X} = \frac{1}{\eta_0}S_c \tag{3.6b}$$

where $\eta_0 = \sqrt{\mu_0/\epsilon_0} = 377\Omega$ is the characteristic impedance of free space and $S_c = c\Delta T/\Delta X$ is known as the Courant number, whose value can be chosen corresponding to the iteration time and grid size. Further, since there is no current flowing in this case, $\sigma = 0$ can be assumed.

3.1.4 Absorbing Boundary Conditions

As mentioned earlier, the update equations won't work for the boundaries. Hence, the boundaries of the electric field grid will remain zero throughout the simulation, which will make the boundaries behave like a perfect electric conductor that reflects all the waves. This is undesirable for the simulation, hence the boundaries need to absorb the waves, which can be done by solving the following equation on the boundary.

$$\begin{aligned}
\frac{\partial^2 E_z}{\partial x^2} - \mu_0\varepsilon_0\frac{\partial^2 E_z}{\partial t^2} &= 0 \\
\left(\frac{\partial^2}{\partial x^2} - \mu_0\varepsilon_0\frac{\partial^2}{\partial t^2}\right) E_z &= 0 \\
\left(\frac{\partial}{\partial x} - \sqrt{\mu_0\varepsilon_0}\frac{\partial}{\partial t}\right)\left(\frac{\partial}{\partial x} + \sqrt{\mu_0\varepsilon_0}\frac{\partial}{\partial t}\right) E_z &= 0 \\
\left(\frac{\partial}{\partial x} - \sqrt{\mu_0\varepsilon_0}\frac{\partial}{\partial t}\right) E_z &= 0
\end{aligned} \tag{3.7}$$

The final form of differential Equation 3.7 can be converted to difference equation at the boundary as follows

$$\frac{\frac{E_z[1,T]+E_z[1,T-1]}{2} - \frac{E_z[0,T]+E_z[0,T-1]}{2}}{\Delta X} - \sqrt{\mu_0\varepsilon_0}\frac{\frac{E_z[0,T]+E_z[1,T]}{2} - \frac{E_z[0,T-1]+E_z[1,T-1]}{2}}{\Delta T} = 0 \tag{3.8}$$

which on algebraic manipulation, gives the following equation

$$E_z[0,T] = E_z[1,T-1] + \frac{S_c - 1}{S_c + 1}\left(E_z[1,T] - E_z[0,T-1]\right) \tag{3.9}$$

3.1.5 Source and Diffuser

The source is defined by the number of grid points involved in one wavelength produced by it. Let this be denoted by N_λ, which is taken as an input in the program. So, $\lambda = N_\lambda \Delta X$,

and the frequency of the source will be given by

$$f_\lambda = \frac{c}{\lambda} = \frac{c}{N_\lambda \Delta X} = \frac{S_c}{N_\lambda \Delta T} \tag{3.10}$$

Here, we express the frequency in terms of known quantities which can be used model the source. The magnitude of the electric field produced by the source can be calculated from Equation 2.22, as the output power and cross-section area of the laser is available from the data sheet. An optical diffuser is implemented as a medium with different refractive index and geometry similar to the grated material in Figure 2.10b.

A C program involving all the aspects discussed in this chapter is given as Appendix A.

Chapter 4

Preliminary Data Analysis

4.1 Data and Requirement

The final part of the internship was preliminary data processing of LiDAR point cloud to initially filter out the noise, and obtain the target. The image was captured in a lab, with the target being the wall in the lab. LeddarTech Pixell LiDAR [10]. This LiDAR has a receiver array of 8×96 and recorded values with resolution of 10 cm as seen below. The data provided was in a text file in the following format. Here x represents the depth, y is the horizontal coordinate and z is the vertical coordinate, as shown in Figure 2.13.

TIME	INDEX	INTENSITY	DISTANCE	X	Y	Z	SUCCESS
280436	96,01	8,760.00	0.14	0.04	-0.14	0.01	1
280436	95,01	10,568.00	0.14	0.04	-0.14	0.01	1
280436	94,01	11,080.00	0.14	0.04	-0.14	0.01	1
280436	93,01	11,852.00	0.15	0.04	-0.15	0.01	1
280436	92,01	11,880.00	0.16	0.05	-0.16	0.01	1

One time instant represented a single image. So each image is supposed to have $8 \times 96 = 768$ points. However the image had more than 900 points. First the extra points were rejected, by choosing a the farthest point corresponding to every index. The final point cloud with required data was stored in a new text file. The C program written to do this is given as Appendix B.1

4.2 Algorithm

The approach for filtering the out the noise was first clustering these points based on the closeness with neighbours. Then rejecting the unwanted clusters.

4.2.1 Clustering

Clustering was done based on closeness of points. Initially geometric discontinuities were found and clusters were divided by these discontinuities. This was done in the following way.

The distance between two point with row can column indices (i_1, j_1) and (i_2, j_2) will be given by

$$dist(i_1, j_1, i_2, j_2) = \sqrt{(x_{i_2,j_2} - x_{i_1,j_1})^2 + (y_{i_2,j_2} - y_{i_1,j_1})^2 + (z_{i_2,j_2} - z_{i_1,j_1})^2} \tag{4.1}$$

For every point P_{ij} where i and j represent the row and column index of the point, the average distance with neighbours was calculated as

$$d_{avg}(i, j) = \frac{dist(i, j, i-1, j) + dist(i, j, i, j-1) + dist(i, j, i+1, j) + dist(i, j, i, j+1)}{4} \tag{4.2}$$

For points on edges, corresponding neighbours are considered, and others neglected. Mean (μ_d) and standard deviation (σ_d) is calculated for all d_{avg}. Then if $d_{avg} > (\mu_d + \sigma_d)$, then the point is called discontinuous. The continuous parts of the image are then clustered where each cluster is bounded by edges of the image or discontinuity.

Then the discontinuous points are reallocated to the clusters. If a point has single discontinuity, which means only one neighbour is $(\mu_d + \sigma_d)$ distance away, then the cluster of the point opposite to the discontinuity is allocated to the discontinuous point. Else if the closest continuous point is closer than $(\mu_d + \sigma_d)$ distance, then its cluster is allocated to the discontinuous point.

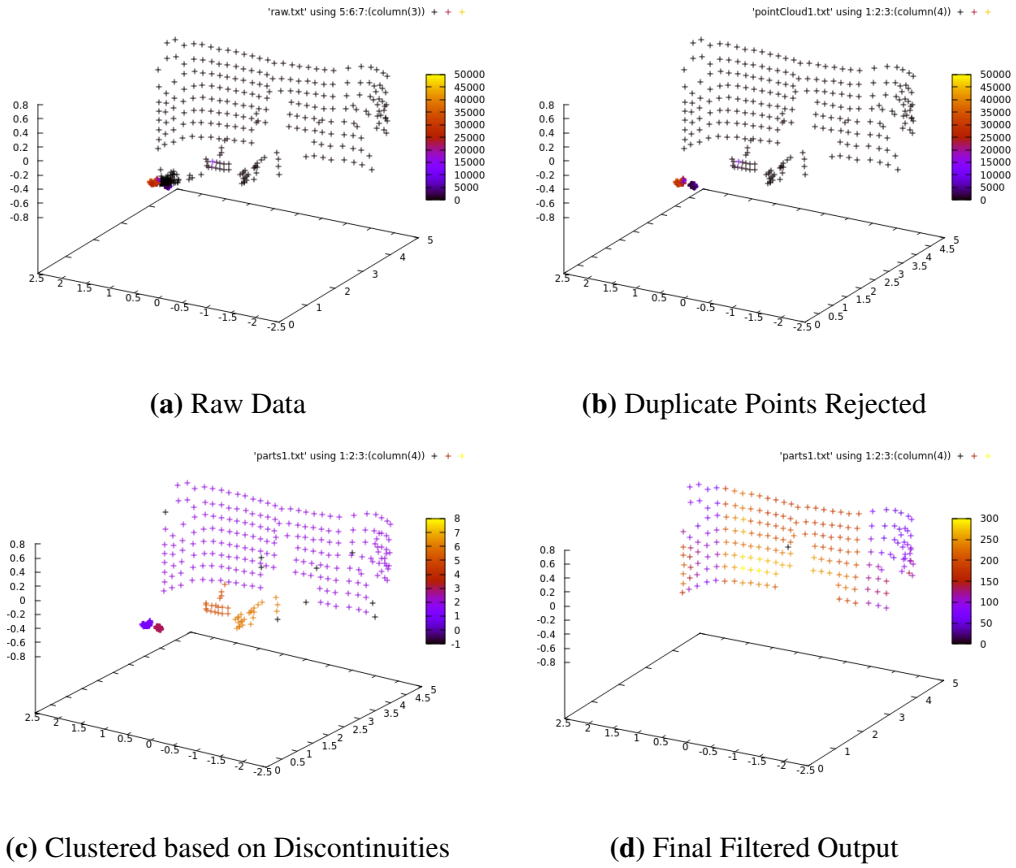
4.2.2 Rejection

Initially, as the resolution of the LiDAR is seen to be 10 cm, the clusters having resolution of the order of 10 cm (less than 20 cm) are rejected as noise. For the remaining points a statistical filter was applied on the x coordinate of the center of mass of these clusters. The mean and standard deviation were calculated as weighted average, with weightage corresponding to number of points in the cluster points allowing only those clusters with

$$(\mu_x - \sigma_x) \leq x_{CoM} \leq (\mu_x + \sigma_x) \tag{4.3}$$

where μ_x and σ_x are the weighted mean and standard deviation of the filtered points respectively, and x_{CoM} is the centre of mass of each cluster. The entire data processing process is

shown in Figure 4.1.



(a) Raw Data

(b) Duplicate Points Rejected

(c) Clustered based on Discontinuities

(d) Final Filtered Output

Figure 4.1: Preliminary Data Processing Results

Chapter 5

Conclusion

5.1 Results

During the first part of my internship, an in-depth study on the physical principles of LiDAR technology was conducted. This included mathematical modeling of these physical concepts. Specifically, the laser and its rate equations, optical diffusers and gratings, and the use of avalanche photodiodes in the receiver array and the mathematical modelling of these components were studied in this part.

The implementation of the source and the diffuser was achieved using the FDTD method, and the results are illustrated in Figure 5.1. The colour represents the magnitude of electric field at each position in the grid.

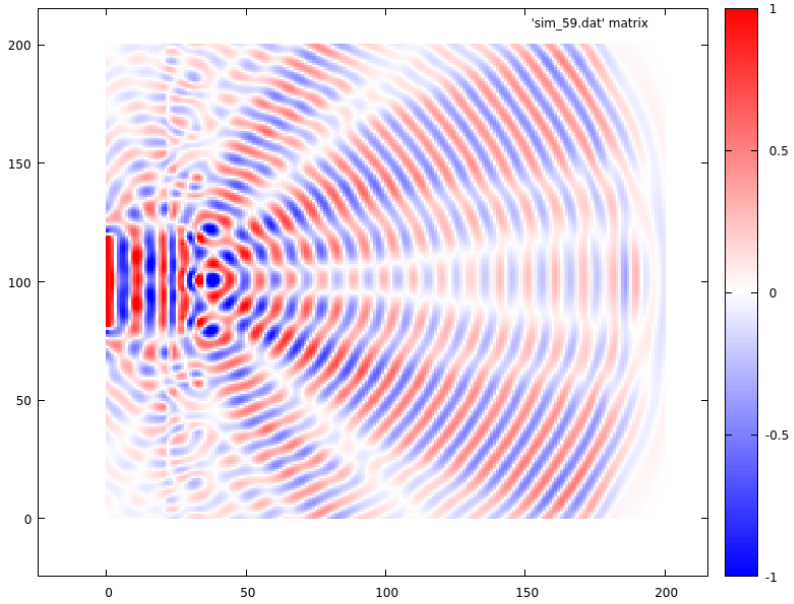
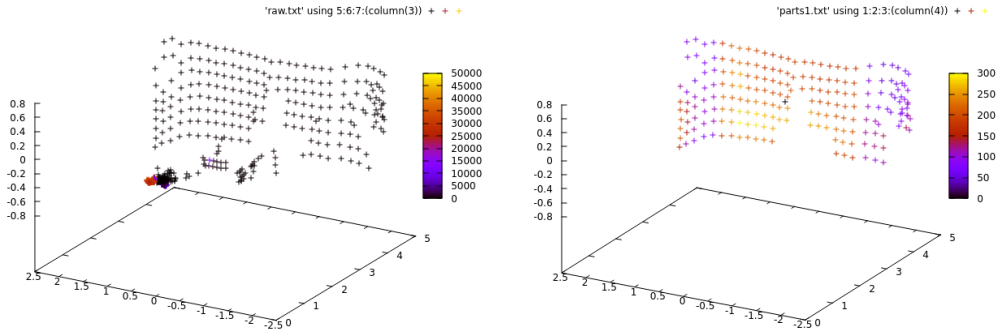


Figure 5.1: FDTD Results

In the final part of the internship, preliminary data processing for a LiDAR image was carried out. This was a completely original work, where closeness-based clustering of points and rejection of unwanted clusters was done. The initial and processed point cloud images are shown in Figure 5.2, where the colour represents intensity. The Center of Mass of the target was estimated to be at coordinates (4.506763 m, 0.413884 m, 0.033824 m), which is indicated by the black point in the figure.



(a) Raw Data **(b)** Final Filtered Output

Figure 5.2: Preliminary Data Processing Results

5.2 Limitations

Despite the results obtained, which are covered in the previous section, there were two main limitations to this internship.

Firstly, the FDTD implementation was conducted solely for the source. The LiDAR system had a range of 50 meters, and simulating a target at this distance was not practically feasible. Due to a lack of understanding about wave propagation, reflection at the target, and the numerical methods required to represent these phenomena, this aspect could not be implemented.

Secondly, the preliminary filtering applied to the LiDAR point cloud image is not a generalized method. It is tailored to images similar to the one used in this project and may not perform well for images with targets of different geometrical properties. This limitation highlights the need for developing more robust and adaptable filtering techniques.

5.3 Future Prospects

The primary future prospects involve addressing the drawbacks and limitations mentioned in the previous section. This includes developing a solution to accurately represent wave propagation in the FDTD simulation. Subsequently, implementing a corresponding array of avalanche photodiodes as the LiDAR sensor will allow for a comprehensive FDTD simulation that incorporates all aspects of the LiDAR system.

Furthermore, there is a need to develop a robust data processing algorithm that can handle images with varying geometrical targets. This algorithm should be adaptable to different scenarios and extendable to other optical sensors beyond LiDAR. Ultimately, these advancements could contribute to improved techniques for pose estimation. Advancing pose estimation capabilities will be quite impactful, considering its applications. Pose estimation enables precise tracking of a target, which opens up a lot of possibilities like following a target accurately, docking maneuvers and other intricate tasks in chaser scenarios, obstacle avoidance, and several other extensive applications.

Bibliography

[1] O. Svelto, D. C. Hanna *et al.*, *Principles of lasers*. Springer, 2010, vol. 1.

[2] K. F. Renk, *Basics of laser physics*. Springer, 2012.

[3] C. of Physics, "Laser rate equation for three-level system | threshold pupming power," https://youtu.be/id7TDGq74V8?si=yDKztBNFBt_r8ZtZ, 2022.

[4] D. J. Griffiths, *Introduction to electrodynamics*. Cambridge University Press, 2023.

[5] E. Hecht, *Optics*. Pearson Education India, 2012.

[6] R. Hui, *Introduction to fiber-optic communications*. Academic Press, 2019.

[7] L. Graham, "The las 1.4 specification." *Photogrammetric Engineering & Remote Sensing*, vol. 78, no. 2, 2012.

[8] X. Li, Y. Shang, B. Hua, R. Yu, and Y. He, "Lidar intensity correction for road marking detection," *Optics and Lasers in Engineering*, vol. 160, p. 107240, 2023.

[9] J. B. Schneider, "Understanding the finite-difference time-domain method," *School of electrical engineering and computer science Washington State University*, vol. 28, 2010.

[10] *Leddar Pixell 3D Flash LiDAR*, LeddarTech, 2021, vesrion 54A0049-5EN. [Online]. Available: https://leddartech.com/app/uploads/dlm_uploads/2021/02/54A0049-5.0EN_Leddar-Pixell-3D-Flash-LiDAR_User-Guide.pdf

Appendix A

FDTD Implementation Code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  #define ALLOC_1D(PNTR, NUM, TYPE) \
6          PNTR=(TYPE *)calloc(NUM, sizeof(TYPE));
7
8  #define ALLOC_2D(PNTR, NUMX, NUMY, TYPE) \
9          PNTR = (TYPE *)calloc((NUMX) * (NUMY), sizeof(TYPE));
10
11 struct Grid {
12     double *hx, *chxh, *chxe;
13     double *hy, *chyh, *chye;
14     double *ez, *ceze, *cezh;
15     int sizeX, sizeY;
16     int time, maxTime;
17     double cdtds;
18 };
19
20 typedef struct Grid Grid;
21
22 #define Hx(M, N) g->hx[(M) * (SizeY-1) + (N)]
23 #define Chxh(M, N) g->chxh[(M) * (SizeY-1) + (N)]
24 #define Chxe(M, N) g->chxe[(M) * (SizeY-1) + (N)]
25
26 #define Hy(M, N) g->hy[(M) * SizeY + (N)]
```

```
27 #define Chyh(M, N) g->chyh[(M) * SizeY + (N)]
28 #define Chye(M, N) g->chye[(M) * SizeY + (N)]
29
30 #define Ez(M, N) g->ez[(M) * SizeY + (N)]
31 #define Ceze(M, N) g->ceze[(M) * SizeY + (N)]
32 #define Cezh(M, N) g->cezh[(M) * SizeY + (N)]
33
34 #define SizeX g->sizeX
35 #define SizeY g->sizeY
36 #define Time g->time
37 #define MaxTime g->maxTime
38 #define Cdtds g->cdtds
39
40 #define EzLeft(M, Q, N) ezLeft[(N) * 6 + (Q) * 3 + (M)]
41 #define EzRight(M, Q, N) ezRight[(N) * 6 + (Q) * 3 + (M)]
42 #define EzTop(N, Q, M) ezTop[(M) * 6 + (Q) * 3 + (N)]
43 #define EzBottom(N, Q, M) ezBottom[(M) * 6 + (Q) * 3 + (N)]
44
45 void gridInit(Grid *g);
46 void abcInit(Grid *g);
47 void abc(Grid *g);
48 void snapshotInit2d(Grid *g);
49 void snapshot2d(Grid *g);
50 void updateE2d(Grid *g);
51 void updateH2d(Grid *g);
52 void sourceInit(Grid *g);
53 double source(double time, double location);
54
55 static double coef0, coef1, coef2;
56 static double *ezLeft, *ezRight, *ezTop, *ezBottom;
57 static int temporalStride = -1, frame = 0, startTime,
58     startNodeX, endNodeX, spatialStrideX,
59     startNodeY, endNodeY, spatialStrideY;
60 static char basename[80] = "sim";
61 static double cdtds, ppw = 0;
62
63 int main() {
```

```
64      Grid *g;
65      ALLOC_1D(g, 1, Grid);
66      gridInit(g);
67      sourceInit(g);
68      abcInit(g);
69      snapshotInit2d(g);
70      for (Time = 0; Time < MaxTime; Time++) {
71          updateH2d(g);
72          updateE2d(g);
73          for (int i = 0; i <= SizeY; i++)
74              if (i > 2 * SizeY / 5 && i < 3 * SizeY / 5)
75                  Ez(1, i) = source(Time, 0.0);
76          abc(g);
77          snapshot2d(g);
78      }
79      return 0;
80  }
81
82  void gridInit(Grid *g) {
83      double imp0 = 377.0;
84      int mm, nn;
85      SizeX = 201;
86      SizeY = 201;
87      MaxTime = 300;
88      Cdtds = 1.0 / sqrt(2.0);
89      double Cdtds2 = 0.66 * Cdtds;
90      ALLOC_2D(g->hx, SizeX, SizeY - 1, double);
91      ALLOC_2D(g->chxh, SizeX, SizeY - 1, double);
92      ALLOC_2D(g->chxe, SizeX, SizeY - 1, double);
93      ALLOC_2D(g->hy, SizeX - 1, SizeY, double);
94      ALLOC_2D(g->chyh, SizeX - 1, SizeY, double);
95      ALLOC_2D(g->chye, SizeX - 1, SizeY, double);
96      ALLOC_2D(g->ez, SizeX, SizeY, double);
97      ALLOC_2D(g->ceze, SizeX, SizeY, double);
98      ALLOC_2D(g->cezh, SizeX, SizeY, double);
99      for (mm = 0; mm < SizeX; mm++)
100         for (nn = 0; nn < SizeY; nn++) {
```

```
101             Ceze(mm, nn) = 1.0;
102             if (mm >= 20 && mm < 35 - 10 * (((nn + 5) % 20) / 10)
                    ) {
103                 Cezh(mm, nn) = Cdtds2 * imp0;
104             } else {
105                 Cezh(mm, nn) = Cdtds * imp0;
106             }
107         }
108     for (mm = 0; mm < SizeX; mm++)
109         for (nn = 0; nn < SizeY - 1; nn++) {
110             Chxh(mm, nn) = 1.0;
111             if (mm >= 20 && mm < 35 - 10 * (((nn + 5) % 20) / 10)
                    ) {
112                 Chxe(mm, nn) = Cdtds2 / imp0;
113             } else {
114                 Chxe(mm, nn) = Cdtds / imp0;
115             }
116         }
117     for (mm = 0; mm < SizeX - 1; mm++)
118         for (nn = 0; nn < SizeY; nn++) {
119             Chyh(mm, nn) = 1.0;
120             if (mm >= 20 && mm < 35 - 10 * (((nn + 5) % 20) / 10)
                    ) {
121                 Chye(mm, nn) = Cdtds2 / imp0;
122             } else {
123                 Chye(mm, nn) = Cdtds / imp0;
124             }
125         }
126     return;
127 }
128
129 void abcInit(Grid *g) {
130     double temp1, temp2;
131     ALLOC_1D(ezLeft, SizeY * 6, double);
132     ALLOC_1D(ezRight, SizeY * 6, double);
133     ALLOC_1D(ezTop, SizeX * 6, double);
134     ALLOC_1D(ezBottom, SizeX * 6, double);
```

```
135     temp1 = sqrt(Cezh(0, 0) * Chye(0, 0));
136     temp2 = 1.0 / temp1 + 2.0 + temp1;
137     coef0 = -(1.0 / temp1 - 2.0 + temp1) / temp2;
138     coef1 = -2.0 * (temp1 - 1.0 / temp1) / temp2;
139     coef2 = 4.0 * (temp1 + 1.0 / temp1) / temp2;
140     return;
141 }
142
143 void abc(Grid *g) {
144     int mm, nn;
145     for (nn = 0; nn < SizeY; nn++) {
146         Ez(0, nn) = coef0 * (Ez(2, nn) + EzLeft(0, 1, nn)) +
147         coef1 * (EzLeft(0, 0, nn) + EzLeft(2, 0, nn) - Ez(1, nn)
              -
148         EzLeft(1, 1, nn)) + coef2 * EzLeft(1, 0, nn) - EzLeft(2,
              1, nn);
149         for (mm = 0; mm < 3; mm++) {
150             EzLeft(mm, 1, nn) = EzLeft(mm, 0, nn);
151             EzLeft(mm, 0, nn) = Ez(mm, nn);
152         }
153     }
154     for (nn = 0; nn < SizeY; nn++) {
155         Ez(SizeX - 1, nn) = coef0 * (Ez(SizeX - 3, nn) + EzRight
              (0, 1, nn)) +
156         coef1 * (EzRight(0, 0, nn) + EzRight(2, 0, nn) - Ez(SizeX
              - 2, nn) -
157         EzRight(1, 1, nn)) + coef2 * EzRight(1, 0, nn) - EzRight
              (2, 1, nn);
158         for (mm = 0; mm < 3; mm++) {
159             EzRight(mm, 1, nn) = EzRight(mm, 0, nn);
160             EzRight(mm, 0, nn) = Ez(SizeX - 1 - mm, nn);
161         }
162     }
163     for (mm = 0; mm < SizeX; mm++) {
164         Ez(mm, 0) = coef0 * (Ez(mm, 2) + EzBottom(0, 1, mm)) +
              coef1 *
165         (EzBottom(0, 0, mm) + EzBottom(2, 0, mm) - Ez(mm, 1) -
```

```
166         EzBottom(1, 1, mm)) + coef2 * EzBottom(1, 0, mm) -
               EzBottom(2, 1, mm);
167         for (nn = 0; nn < 3; nn++) {
168             EzBottom(nn, 1, mm) = EzBottom(nn, 0, mm);
169             EzBottom(nn, 0, mm) = Ez(mm, nn);
170         }
171     }
172     for (mm = 0; mm < SizeX; mm++) {
173         Ez(mm, SizeY - 1) = coef0 * (Ez(mm, SizeY - 3) + EzTop(0,
                1, mm)) +
174         coef1 * (EzTop(0, 0, mm) + EzTop(2, 0, mm) - Ez(mm, SizeY
                - 2) -
175         EzTop(1, 1, mm)) + coef2 * EzTop(1, 0, mm) - EzTop(2, 1,
               mm);
176         for (nn = 0; nn < 3; nn++) {
177             EzTop(nn, 1, mm) = EzTop(nn, 0, mm);
178             EzTop(nn, 0, mm) = Ez(mm, SizeY - 1 - nn);
179         }
180     }
181     return;
182 }
183
184 void snapshotInit2d(Grid *g) {
185     printf("Duration of simulation is %d steps.\n", MaxTime);
186     printf("Enter start time and frame interval: ");
187     scanf(" %d %d", &startTime, &temporalStride);
188     startNodeX = 0; endNodeX = SizeX - 1; spatialStrideX = 1;
189     startNodeY = 0; endNodeY = SizeY - 1; spatialStrideY = 1;
190     return;
191 }
192
193 void snapshot2d(Grid *g) {
194     int mm, nn;
195     float temp;
196     char filename[100];
197     FILE *out;
198     if (temporalStride == -1) {
```

```
199         return;
200     }
201     if (Time >= startTime && (Time - startTime) % temporalStride
           == 0) {
202         sprintf(filename, "%s_%d.dat", basename, frame++);
203         out = fopen(filename, "w");
204         for (nn = endNodeY; nn >= startNodeY; nn -=
               spatialStrideY) {
205             for (mm = startNodeX; mm <= endNodeX; mm +=
                   spatialStrideX) {
206                 temp = (float)Ez(mm, nn);
207                 fprintf(out, "%f ", temp);
208             }
209             fprintf(out, "\n");
210         }
211         fclose(out);
212     }
213     return;
214 }
215
216 void updateH2d(Grid *g) {
217     int mm, nn;
218     for (mm = 0; mm < SizeX; mm++)
219         for (nn = 0; nn < SizeY - 1; nn++)
220             Hx(mm, nn) = Chxh(mm, nn) * Hx(mm, nn) -
221             Chxe(mm, nn) * (Ez(mm, nn + 1) - Ez(mm, nn));
222     for (mm = 0; mm < SizeX - 1; mm++)
223         for (nn = 0; nn < SizeY; nn++)
224             Hy(mm, nn) = Chyh(mm, nn) * Hy(mm, nn) +
225             Chye(mm, nn) * (Ez(mm + 1, nn) - Ez(mm, nn));
226     return;
227 }
228
229 void updateE2d(Grid *g) {
230     int mm, nn;
231     for (mm = 1; mm < SizeX - 1; mm++)
232         for (nn = 1; nn < SizeY - 1; nn++)
```

```
233             Ez(mm, nn) = Ceze(mm, nn) * Ez(mm, nn) + Cezh(mm, nn)
                    * ((Hy(mm, nn)
234             - Hy(mm - 1, nn)) - (Hx(mm, nn) - Hx(mm, nn - 1)));
235     return;
236 }
237
238 void sourceInit(Grid *g){
239     printf("Enter␣the␣points␣per␣wavelength␣for␣source:␣");
240     scanf("␣%lf", &ppw);
241     cdtds = Cdtds;
242     return;
243 }
244
245 double source(double time, double location) {
246     double arg;
247     if (ppw <= 0) {
248         fprintf(stderr,"Points␣per␣wavelength␣must␣be␣positive.\n
               ");
249         exit(-1);
250     }
251     arg = M_PI * ((cdtds * time - location) / ppw - 1.0);
252     return cos(2 * arg);
253 }
```

Appendix B

Preliminary Data Processing

B.1 C Program to convert given data to point cloud images

```
#include <stdio.h>
#include <stdlib.h>

#define index ((96*(row-1))+column-1)

int main() {
    char *inten, *inputfile, *outputfile;
    inten = (char *)calloc(20,sizeof(char));
    inputfile = (char *)calloc(20,sizeof(char));
    outputfile = (char *)calloc(20,sizeof(char));
    int f, j = 1, row, column;
    long int time, timeref;
    float intensity[768], x[768], y[768], z[768], dist[768] =
       {0.00};
    float intensity_t, x_t, y_t, z_t, dist_t;
    printf("Enter name of input file: ");
    scanf("%s", inputfile);
    FILE* file = fopen(inputfile, "r");
    if (!file) {
        perror("Error opening file");
        return 1;
    }
```

```
22      fscanf(file, "%ld␣%s", &timeref, inten);
23      fclose(file);
24      FILE* filer = fopen(inputfile, "r");
25      if (!filer) {
26          perror("Error␣opening␣file");
27          return 1;
28      }
29      while (fscanf(filer, "%ld␣%d,%d␣%s␣%f␣%f␣%f␣%f␣%d", &time, &
           column,
30      &row, inten, &dist_t, &x_t, &y_t, &z_t, &f) != EOF) {
31          for (int j = 0; inten[j]; j++) {
32              if (inten[j] == ',') {
33                  for (int k = j; inten[k]; k++) {
34                      inten[k] = inten[k + 1];
35                  }
36              }
37          }
38          sscanf(inten, "%f", &intensity_t);
39          if (dist_t > dist[index]) {
40              intensity[index] = intensity_t;
41              dist[index] = dist_t;
42              x[index] = x_t;
43              y[index] = y_t;
44              z[index] = z_t;
45          }
46          if (time != timeref) {
47              sprintf(outputfile, "pointCloud%d.txt", j);
48              FILE* filew = fopen(outputfile, "w");
49              double x_tot = 0, y_tot = 0, z_tot = 0, i_tot = 0;
50              for (int i = 0; i < 768; i++) {
51                  fprintf(filew, "%f␣%f␣%f␣%f\n", x[i], y[i], z[i],
                       intensity[i]);
52                  dist[i] = -1.00;
53                  x_tot += x[i]; y_tot += y[i]; z_tot = z[i];
54              }
55              fclose(filew);
56              timeref = time;
```

```
57              j += 1;
58          }
59      }
60      fclose(filer);
61      return 0;
62  }
```

B.2 C Program for clustering and filtering

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  #define INDEX(row, column) ((96*((row)-1))+(column)-1)
6
7  struct pointCloud {
8      float *x, *y, *z, *intensity;
9  };
10
11 typedef struct pointCloud pointCloud;
12
13 float distance (pointCloud *pc, int i1, int i2);
14 void computeAvgDist (pointCloud *pc, float *avgDist);
15 int divideParts (int *parts);
16 int removeDiscontinuity (pointCloud *pc, int *parts, float mean,
       float stdDev);
17
18 int main() {
19      pointCloud *pc1;
20      pc1 = (pointCloud *)calloc(1,sizeof(pointCloud));
21      pc1->x = (float *)calloc(768,sizeof(float));
22      pc1->y = (float *)calloc(768,sizeof(float));
23      pc1->z = (float *)calloc(768,sizeof(float));
24      pc1->intensity = (float *)calloc(768,sizeof(float));
25      int filenum = 1;
26      char *fileread, *filewrite;
```

```
27      fileread = (char *)calloc(20,sizeof(char));
28      filewrite = (char *)calloc(20,sizeof(char));
29      sprintf(fileread, "pointCloud%d.txt", filenum);
30      sprintf(filewrite, "parts%d.txt", filenum);
31      FILE* filer = fopen(fileread, "r");
32      if (!filer) {
33          perror("Error opening file");
34          return 1;
35      }
36      int i = 0;
37      while (fscanf(filer, "%f %f %f %f", pc1->x+i, pc1->y+i,
38      pc1->z+i, pc1->intensity+i) != EOF) {
39          i += 1;
40      }
41      fclose(filer);
42      float *avgDist;
43      int *parts;
44      avgDist = (float *)calloc(768,sizeof(float));
45      parts = (int *)calloc(768,sizeof(int));
46      computeAvgDist(pc1, avgDist);
47      float mean_dist = 0, var_dist = 0, stdDev_dist;
48      float mean_int = 0, var_int = 0, stdDev_int;
49      float mean_x = 0, var_x = 0, stdDev_x;
50      float x_com = 0, y_com = 0, z_com = 0, int_com = 0;
51      float threshold = 0.05;
52      for (i = 0; i < 768; i++) {
53          mean_dist += avgDist[i];
54      }
55      mean_dist = mean_dist / 768;
56      for (i = 0; i < 768; i++) {
57          var_dist += ((avgDist[i] - mean_dist) * (avgDist[i] -
               mean_dist));
58      }
59      var_dist = var_dist / 768;
60      stdDev_dist = sqrt(var_dist);
61          for (i = 0; i < 768; i++) {
62          if(avgDist[i] > (mean_dist + stdDev_dist) ||
```

```
63          avgDist[i] < (mean_dist - stdDev_dist)) {
64              parts[i] = -1;
65          } else {
66              parts[i] = 0;
67          }
68      }
69      int k = divideParts(parts);
70      while (removeDiscontinuity (pc1, parts, mean_dist,
           stdDev_dist)) {}
71      float *meanByParts, *meanXcoordinate;
72      int *count;
73      meanByParts = (float *)calloc(k,sizeof(float));
74      meanXcoordinate = (float *)calloc(k,sizeof(float));
75      count = (int *)calloc(k,sizeof(int));
76      for (i = 0; i < k; i++) {
77          meanByParts[i] = 0.00;
78          meanXcoordinate[i] = 0.00;
79          count[i] = 0;
80      }
81      for (i = 0; i < 768; i++) {
82          if (parts[i] != -1) {
83              meanByParts[parts[i]] += avgDist[i];
84              meanXcoordinate[parts[i]] += pc1->x[i];
85              count[parts[i]] += 1;
86          }
87      }
88      for (i = 1; i < k; i++) {
89          meanByParts[i] = meanByParts[i] / count[i];
90          meanXcoordinate[i] = meanXcoordinate[i] / count[i];
91      }
92      k = 0;
93      for (i = 0; i < 768; i++) {
94          if (meanByParts[parts[i]] > threshold) {
95              mean_int += pc1->intensity[i];
96              mean_x += pc1->x[i];
97              k += 1;
98          }
```

```
99      }
100     mean_int = mean_int / k;
101     mean_x = mean_x / k;
102     for (i = 0; i < 768; i++) {
103         if (meanByParts[parts[i]] > threshold) {
104             var_int += ((pc1->intensity[i] - mean_int) * (pc1->
                    intensity[i] - mean_int));
105             var_x += ((pc1->x[i] - mean_x) * (pc1->x[i] - mean_x)
                    );
106         }
107     }
108     var_int = var_int / k;
109     var_x = var_x / k;
110     stdDev_int = sqrt(var_int);
111     stdDev_x = sqrt(var_x);
112     FILE* filew = fopen(filewrite, "w");
113     for (i = 0; i < 768; i++) {
114         if (meanByParts[parts[i]] > threshold) {
115         if (meanXcoordinate[parts[i]] > (mean_x - stdDev_x) &&
116         meanXcoordinate[parts[i]] < (mean_x + stdDev_x)) {
117             fprintf(filew, "%f␣%f␣%f␣%f␣%d␣%d\n", pc1->x[i], pc1
                    ->y[i],
118             pc1->z[i], pc1->intensity[i], i/96, i%96);
119             x_com += pc1->x[i] * pc1->intensity[i];
120             y_com += pc1->y[i] * pc1->intensity[i];
121             z_com += pc1->z[i] * pc1->intensity[i];
122             int_com += pc1->intensity[i];
123         }
124         }
125     }
126     x_com = x_com / int_com;
127     y_com = y_com / int_com;
128     z_com = z_com / int_com;
129     fprintf(filew, "%f␣%f␣%f␣%f␣%d␣%d\n", x_com, y_com, z_com,
           0.00, 0, 0);
130     fclose(filew);
131     return 0;
```

```
132 }
133
134 float distance (pointCloud *pc, int i1, int i2) {
135     float distsqr = (pc->x[i1] - pc->x[i2]) * (pc->x[i1] - pc->x[
        i2]) +
136     (pc->y[i1] - pc->y[i2]) * (pc->y[i1] - pc->y[i2]) +
137     (pc->z[i1] - pc->z[i2]) * (pc->z[i1] - pc->z[i2]);
138     return sqrt(distsqr);
139 }
140
141 void computeAvgDist (pointCloud *pc, float *avgDist) {
142     avgDist[INDEX(1,1)] = ((distance(pc, INDEX(1,1), INDEX(1,2))
        +
143     distance(pc, INDEX(1,1), INDEX(2,1))) / 2);
144     avgDist[INDEX(1,96)] = ((distance(pc, INDEX(1,96), INDEX
        (1,95)) +
145     distance(pc, INDEX(1,96), INDEX(2,96))) / 2);
146     avgDist[INDEX(8,1)] = ((distance(pc, INDEX(8,1), INDEX(8,2))
        +
147     distance(pc, INDEX(8,1), INDEX(7,1))) / 2);
148     avgDist[INDEX(8,96)] = ((distance(pc, INDEX(8,96), INDEX
        (7,96)) +
149     distance(pc, INDEX(8,96), INDEX(8,95))) / 2);
150     for (int i = 2; i < 8; i++) {
151         avgDist[INDEX(i,1)] = ((distance(pc, INDEX(i,1), INDEX(i
            -1,1)) +
152         distance(pc, INDEX(i,1), INDEX(i+1,1)) +
153         distance(pc, INDEX(i,1), INDEX(i,2))) / 3);
154         avgDist[INDEX(i,96)] = ((distance(pc, INDEX(i,96), INDEX(
            i-1,96)) +
155         distance(pc, INDEX(i,96), INDEX(i+1,96)) +
156         distance(pc, INDEX(i,96), INDEX(i,95))) / 3);
157     }
158     for (int i = 2; i < 96; i++) {
159         avgDist[INDEX(1,i)] = ((distance(pc, INDEX(1,i), INDEX(1,
            i-1)) +
160         distance(pc, INDEX(1,i), INDEX(1,i+1)) +
```

```
161         distance(pc, INDEX(1,i), INDEX(2,i))) / 3);
162         avgDist[INDEX(8,i)] = ((distance(pc, INDEX(8,i), INDEX(8,
               i-1)) +
163         distance(pc, INDEX(8,i), INDEX(8,i+1)) +
164         distance(pc, INDEX(8,i), INDEX(7,i))) / 3);
165         }
166         for (int i = 2; i < 8; i++) {
167                 for (int j = 2; j < 96; j++) {
168             avgDist[INDEX(i,j)] = ((distance(pc, INDEX(i,j),
                   INDEX(i-1,j)) +
169             distance(pc, INDEX(i,j), INDEX(i+1,j)) +
170             distance(pc, INDEX(i,j), INDEX(i,j-1)) +
171             distance(pc, INDEX(i,j), INDEX(i,j+1))) / 4);
172         }
173     }
174 }
175
176 int divideParts (int *parts) {
177     int k = 1;
178     for (int i = 1; i <= 8; i++) {
179         for (int j = 1; j <= 96; j++) {
180             if (parts[INDEX(i,j)] != -1) {
181                 if (i == 1) {
182                     if (j == 1) {
183                         parts[INDEX(i,j)] = k;
184                     } else if (parts[INDEX(i,j-1)] != -1) {
185                         parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
186                     } else {
187                         k += 1;
188                         parts[INDEX(i,j)] = k;
189                     }
190                 } else {
191                     if (j == 1) {
192                         if (parts[INDEX(i-1,j)] != -1) {
193                             parts[INDEX(i,j)] = parts[INDEX(i-1,j
                                   )];
194                         } else {
```

```
195                             k += 1;
196                             parts[INDEX(i,j)] = k;
197                         }
198                     } else if (parts[INDEX(i-1,j)] != -1) {
199                         parts[INDEX(i,j)] = parts[INDEX(i-1,j)];
200                     } else if (parts[INDEX(i,j-1)] != -1) {
201                         parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
202                     } else {
203                         k += 1;
204                         parts[INDEX(i,j)] = k;
205                     }
206                 }
207             }
208         }
209     }
210     for (int i = 8; i >= 1; i--) {
211         for (int j = 96; j > 1; j--) {
212             if (parts[INDEX(i,j-1)] != -1 && parts[INDEX(i,j)] !=
                    -1 &&
213             parts[INDEX(i,j-1)] != parts[INDEX(i,j)]) {
214                 parts[INDEX(i,j-1)] = parts[INDEX(i,j)];
215             }
216         }
217     }
218     return k;
219 }
220
221 int removeDiscontinuity (pointCloud *pc, int *parts, float mean,
       float stdDev) {
222     int change = 0;
223     float up, down, left, right;
224     for (int i = 1; i <= 8; i++) {
225         for (int j = 1; j <= 96; j++) {
226             if (parts[INDEX(i,j)] == -1) {
227                 if (i == 1 && j == 1) {
228                     down = distance (pc, INDEX(i,j), INDEX(i+1,j)
                            );
```

```
229                     right = distance (pc, INDEX(i,j), INDEX(i,j
                           +1));
230                     if (down <= right && parts[INDEX(i+1,j)] !=
                           -1) {
231                         parts[INDEX(i,j)] = parts[INDEX(i+1,j)];
232                         change = 1;
233                     } else if (parts[INDEX(i,j+1)] != -1) {
234                         parts[INDEX(i,j)] = parts[INDEX(i,j+1)];
235                         change = 1;
236                     }
237                 } else if (i == 1 && j == 96) {
238                     down = distance (pc, INDEX(i,j), INDEX(i+1,j)
                           );
239                     left = distance (pc, INDEX(i,j), INDEX(i,j-1)
                           );
240                     if (down <= left && parts[INDEX(i+1,j)] !=
                           -1) {
241                         parts[INDEX(i,j)] = parts[INDEX(i+1,j)];
242                         change = 1;
243                     } else if (parts[INDEX(i,j-1)] != -1) {
244                         parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
245                         change = 1;
246                     }
247                 } else if (i == 8 && j == 1) {
248                     up = distance (pc, INDEX(i,j), INDEX(i-1,j));
249                     right = distance (pc, INDEX(i,j), INDEX(i,j
                           +1));
250                     if (up <= right && parts[INDEX(i-1,j)] != -1)
                           {
251                         parts[INDEX(i,j)] = parts[INDEX(i-1,j)];
252                         change = 1;
253                     } else if (parts[INDEX(i,j+1)] != -1) {
254                         parts[INDEX(i,j)] = parts[INDEX(i,j+1)];
255                         change = 1;
256                     }
257                 } else if (i == 8 && j == 96) {
258                     up = distance (pc, INDEX(i,j), INDEX(i-1,j));
```

```
259                     left = distance (pc, INDEX(i,j), INDEX(i,j-1)
                            );
260                     if (up <= left && parts[INDEX(i-1,j)] != -1)
                            {
261                         parts[INDEX(i,j)] = parts[INDEX(i-1,j)];
262                         change = 1;
263                     } else if (parts[INDEX(i,j-1)] != -1) {
264                         parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
265                         change = 1;
266                     }
267                 } else if (i == 1) {
268                     down = distance (pc, INDEX(i,j), INDEX(i+1,j)
                            );
269                     left = distance (pc, INDEX(i,j), INDEX(i,j-1)
                            );
270                     right = distance (pc, INDEX(i,j), INDEX(i,j
                            +1));
271                     if (down <= (mean + stdDev) && left <= (mean
                            + stdDev) &&
272                     right >= (mean + stdDev) && parts[INDEX(i,j
                            -1)] != -1) {
273                         parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
274                         change = 1;
275                     } else if (down <= (mean + stdDev) && left >=
                             (mean + stdDev) &&
276                     right <= (mean + stdDev) && parts[INDEX(i,j
                            +1)] != -1) {
277                         parts[INDEX(i,j)] = parts[INDEX(i,j+1)];
278                         change = 1;
279                     }
280                 } else if (i == 8) {
281                     up = distance (pc, INDEX(i,j), INDEX(i-1,j));
282                     left = distance (pc, INDEX(i,j), INDEX(i,j-1)
                            );
283                     right = distance (pc, INDEX(i,j), INDEX(i,j
                            +1));
284                     if (up <= (mean + stdDev) && left <= (mean +
```

```
                    stdDev) &&
285                 right >= (mean + stdDev) && parts[INDEX(i,j
                    -1)] != -1) {
286                  parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
287                  change = 1;
288                 } else if (up <= (mean + stdDev) && left >= (
                    mean + stdDev) &&
289                 right <= (mean + stdDev) && parts[INDEX(i,j
                    +1)] != -1) {
290                  parts[INDEX(i,j)] = parts[INDEX(i,j+1)];
291                  change = 1;
292                 }
293             } else if (j == 1) {
294                 up = distance (pc, INDEX(i,j), INDEX(i-1,j));
295                 down = distance (pc, INDEX(i,j), INDEX(i+1,j)
                    );
296                 right = distance (pc, INDEX(i,j), INDEX(i,j
                    +1));
297                 if (up <= (mean + stdDev) && down >= (mean +
                    stdDev) &&
298                 right <= (mean + stdDev) && parts[INDEX(i-1,j
                    )] != -1) {
299                  parts[INDEX(i,j)] = parts[INDEX(i-1,j)];
300                  change = 1;
301                 } else if (up >= (mean + stdDev) && down <= (
                    mean + stdDev) &&
302                 right <= (mean + stdDev) && parts[INDEX(i+1,j
                    )] != -1) {
303                  parts[INDEX(i,j)] = parts[INDEX(i+1,j)];
304                  change = 1;
305                 }
306             } else if (j == 96) {
307                 up = distance (pc, INDEX(i,j), INDEX(i-1,j));
308                 down = distance (pc, INDEX(i,j), INDEX(i+1,j)
                    );
309                 left = distance (pc, INDEX(i,j), INDEX(i,j-1)
                    );
```

```
310                     if (up <= (mean + stdDev) && down >= (mean +
                           stdDev) &&
311                     left <= (mean + stdDev) && parts[INDEX(i-1,j)
                           ] != -1) {
312                         parts[INDEX(i,j)] = parts[INDEX(i-1,j)];
313                         change = 1;
314                     } else if (up >= (mean + stdDev) && down <= (
                           mean + stdDev) &&
315                     left <= (mean + stdDev) && parts[INDEX(i+1,j)
                           ] != -1) {
316                         parts[INDEX(i,j)] = parts[INDEX(i+1,j)];
317                         change = 1;
318                     }
319                 } else {
320                     up = distance (pc, INDEX(i,j), INDEX(i-1,j));
321                     down = distance (pc, INDEX(i,j), INDEX(i+1,j)
                           );
322                     left = distance (pc, INDEX(i,j), INDEX(i,j-1)
                           );
323                     right = distance (pc, INDEX(i,j), INDEX(i,j
                           +1));
324                     if (up <= (mean + stdDev) && down >= (mean +
                           stdDev) &&
325                     left <= (mean + stdDev) && right <= (mean +
                           stdDev) &&
326                     parts[INDEX(i-1,j)] != -1) {
327                         parts[INDEX(i,j)] = parts[INDEX(i-1,j)];
328                         change = 1;
329                     } else if (up >= (mean + stdDev) && down <= (
                           mean + stdDev) &&
330                     left <= (mean + stdDev) && right <= (mean +
                           stdDev) &&
331                     parts[INDEX(i+1,j)] != -1) {
332                         parts[INDEX(i,j)] = parts[INDEX(i+1,j)];
333                         change = 1;
334                     } else if (up <= (mean + stdDev) && down <= (
                           mean + stdDev) &&
```

```
335                     left <= (mean + stdDev) && right >= (mean +
                            stdDev) &&
336                     parts[INDEX(i,j-1)] != -1) {
337                         parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
338                         change = 1;
339                     } else if (up <= (mean + stdDev) && down <= (
                            mean + stdDev) &&
340                     left >= (mean + stdDev) && right <= (mean +
                            stdDev) &&
341                     parts[INDEX(i,j+1)] != -1) {
342                         parts[INDEX(i,j)] = parts[INDEX(i,j+1)];
343                         change = 1;
344                     } else if (up <= (mean + stdDev) && parts[
                            INDEX(i-1,j)] != -1) {
345                         parts[INDEX(i,j)] = parts[INDEX(i-1,j)];
346                         change = 1;
347                     } else if (down <= (mean + stdDev) && parts[
                            INDEX(i+1,j)] != -1) {
348                         parts[INDEX(i,j)] = parts[INDEX(i+1,j)];
349                         change = 1;
350                     } else if (left <= (mean + stdDev) && parts[
                            INDEX(i,j-1)] != -1) {
351                         parts[INDEX(i,j)] = parts[INDEX(i,j-1)];
352                         change = 1;
353                     } else if (right <= (mean +stdDev) && parts[
                            INDEX(i,j+1)] != -1) {
354                         parts[INDEX(i,j)] = parts[INDEX(i,j+1)];
355                         change = 1;
356                     }
357                 }
358             }
359         }
360     }
361     return change;
362 }
```